AI Candidate Assessment Platform: MVP Edition

Architecture, System Design and API Contracts

Multi-tenant, built in 3 to 4 days on Azure AI Foundry, SQL, Blob, Communication Services and Container Apps

Prepared October 31, 2026 | Version 1.0 MVP | Confidential, internal use

What this document gives you

A final MVP design that a small team can build in 3 to 4 days and use for real hiring. It keeps the useful parts of the full design (multi-tenancy, Azure AI Foundry, SQL, Blob, email, server-side exam timer, PDF and CSV export, GitHub Actions deploy) and removes everything that is not needed yet. Every part starts with a picture, then a short explanation.

#	Section
1	MVP scope: what is in and what is out
2	Overall architecture
3	System design: how a request and a background job run
4	Multi-tenancy
5	How frontend and backend talk to each other
6	Data layer
7	AI layer
8	Other layers: Blob, email, job loop
9	API contracts, one diagram per group
10	Final API contract map
11	CI/CD with GitHub Actions to Azure Container Apps
12	4-day build plan, setup and risks

Colour guide: blue = recruiter screens and UI, green = candidate and data, teal = API, amber = background work, purple = AI and email services, gray = infrastructure, red = failures, limits and secrets.

1. MVP scope: what is in and what is out

Goal: several recruiters from different teams can create assessments, invite candidates, and read results at the same time, on one small deployment.

Included (Yes)	Left out for now (No)
Multi-tenant: `tenant_id` on every table, tenant taken from the session	Row-Level Security, per-tenant quotas UI
Login with session cookie (no JWT); roles Admin and Recruiter; candidates use a one-time link	SSO, MFA, CSRF tokens (SameSite cookie is used)
Job description by **paste or file upload** (PDF or TXT, stored in Blob Storage)	Large file types, virus scan
AI question generation (MCQ, descriptive, cognitive) with JSON check and 3 retries	Prompt versioning, circuit breaker
Review, edit, regenerate, add own questions, approve	Question bank across assessments
Participants by email list or CSV; invitation emails via Azure Communication Services; resend	Delivery webhooks, bounce handling
Candidate exam: Start button, server deadline, autosave every 30 s, safe submit, auto-submit when time ends	Proctoring, tab-switch tracking
Auto-grading of MCQ, AI marking of descriptive answers, reviewer override	Fairness reports, rubric analytics
Results table, candidate detail, **CSV and PDF export**	Candidate comparison view (add later)
GitHub Actions to Azure Container Registry to Azure Container Apps	Staging environment, infrastructure as code
Application Insights logs, short audit table	Redis, Service Bus, Front Door, read replicas

MVP limits (protect cost and keep things simple)

Limit	Value
Job description	20,000 characters, at least 100 words; file up to 2 MB (PDF or TXT)
Questions per assessment	40 total (max 20 per section)
Participants per assessment	200
AI generation calls	20 per tenant per day (counted from the jobs table)
AI marking	up to 3 calls at a time, each retried up to 3 times
Login	5 wrong passwords in 10 minutes locks that email for 10 minutes
Sessions	idle 30 minutes, absolute 8 hours; candidate link valid 7 days

2. Overall architecture

Two containers run in one Azure Container Apps environment. **web** is the Next.js app with the staff screens and the candidate portal. **api** is the backend; it also runs a small job loop that calls GPT-4o and sends emails. Everything else is a managed Azure service.

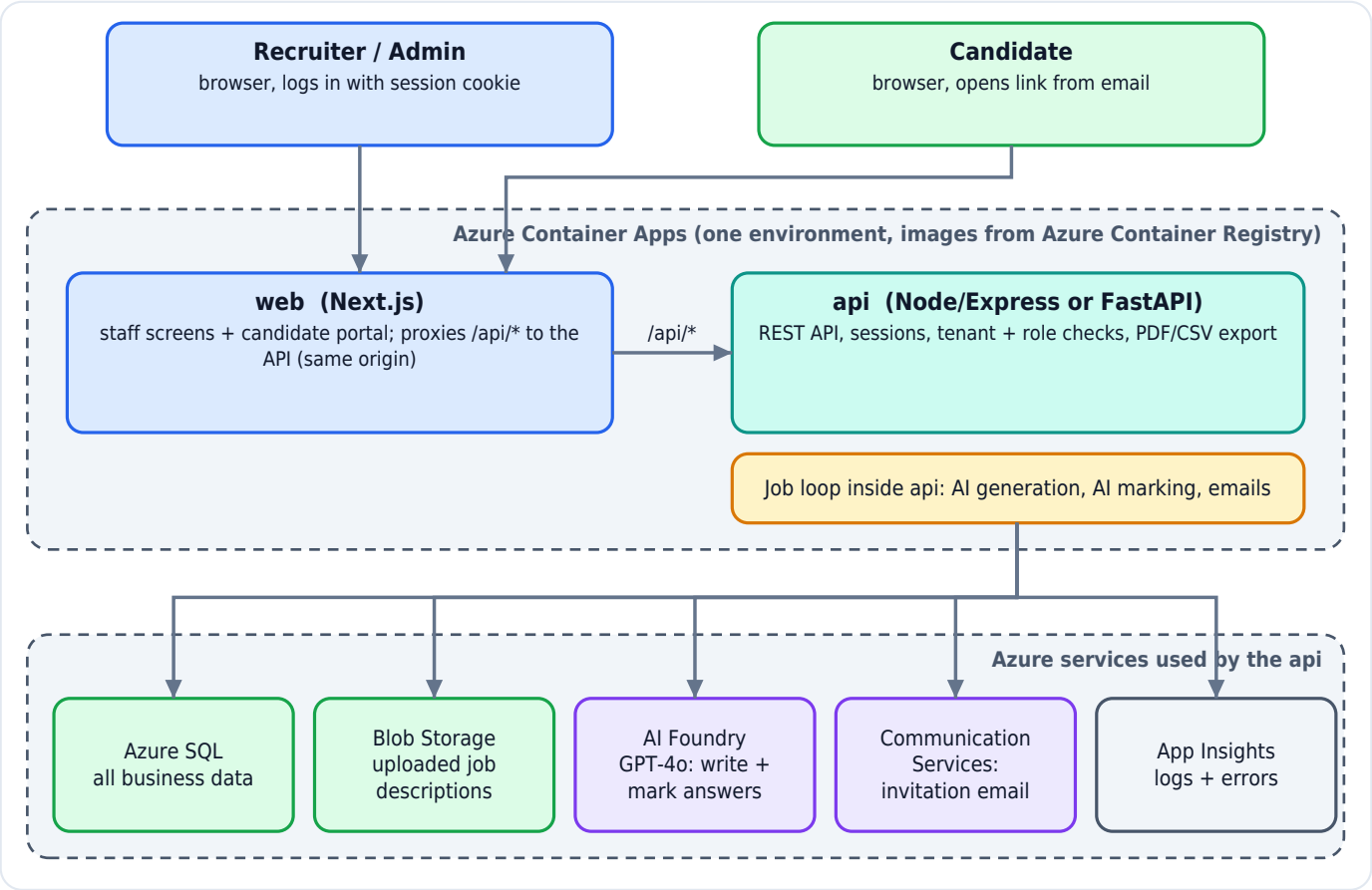


Figure 1. Overall architecture. Arrows show who calls whom.

Part	Role in the MVP
web (Next.js)	All screens. Forwards `/api/*` to the api container so the browser sees one origin.
api	REST API, login sessions, tenant and role checks, JD text extraction, PDF and CSV export, job loop.
Azure SQL	All business data, sessions, jobs, audit log.
Blob Storage	Original uploaded job description files, one folder per tenant.
Azure AI Foundry (GPT-4o)	Writes questions and marks descriptive answers.
Communication Services	Sends invitation emails with the candidate link.
Container Registry + Container Apps	Stores and runs the two images. Deployed by GitHub Actions.
Application Insights	Request logs and errors. Switched on with one setting.

3. System design: request path and background path

Anything fast (login, lists, autosave) is answered inside the request. Anything slow (GPT-4o, email) becomes a row in the `jobs` table; a loop inside the api container picks it up, does the work, and writes the result. The screen polls GET `/jobs/{id}` every 2 seconds. This replaces a message queue and keeps the MVP small.

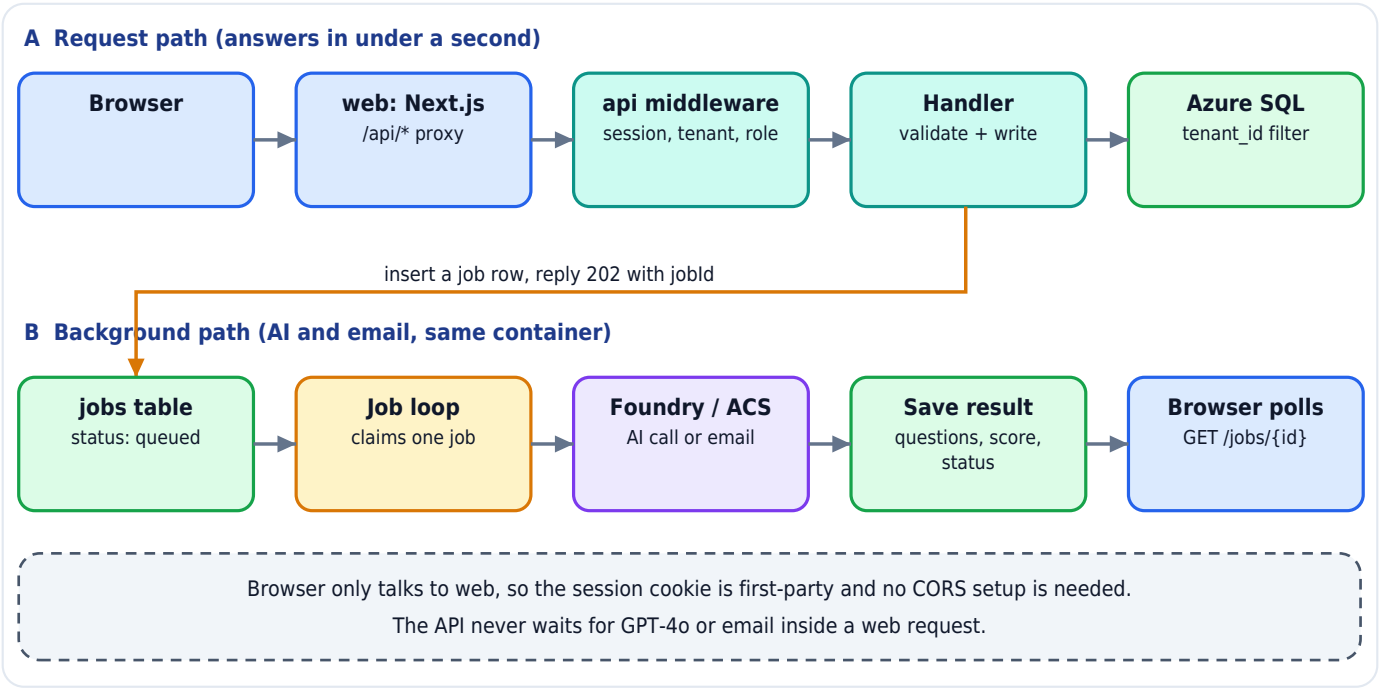


Figure 2. Two paths: fast requests (A) and slow AI or email work (B).

- **Job claiming:** the loop takes a job with `UPDATE ... SET status='running' WHERE status='queued'` inside a transaction, so two api replicas never run the same job.
- **Retries:** a failed job is retried up to 3 times, then marked `failed` with a message the screen can show.
- **Restart safety:** on start the loop re-queues jobs stuck in running for more than 5 minutes.
- **Deadline timer:** the same loop runs every minute and submits attempts whose deadline has passed.
- **Scaling:** start with 1 api replica (enough for 15 to 20 users); the design allows 2 or more.

4. Multi-tenancy

A tenant is one team or company using the platform. All tenants share the same app and database. Isolation comes from one rule that is applied everywhere.

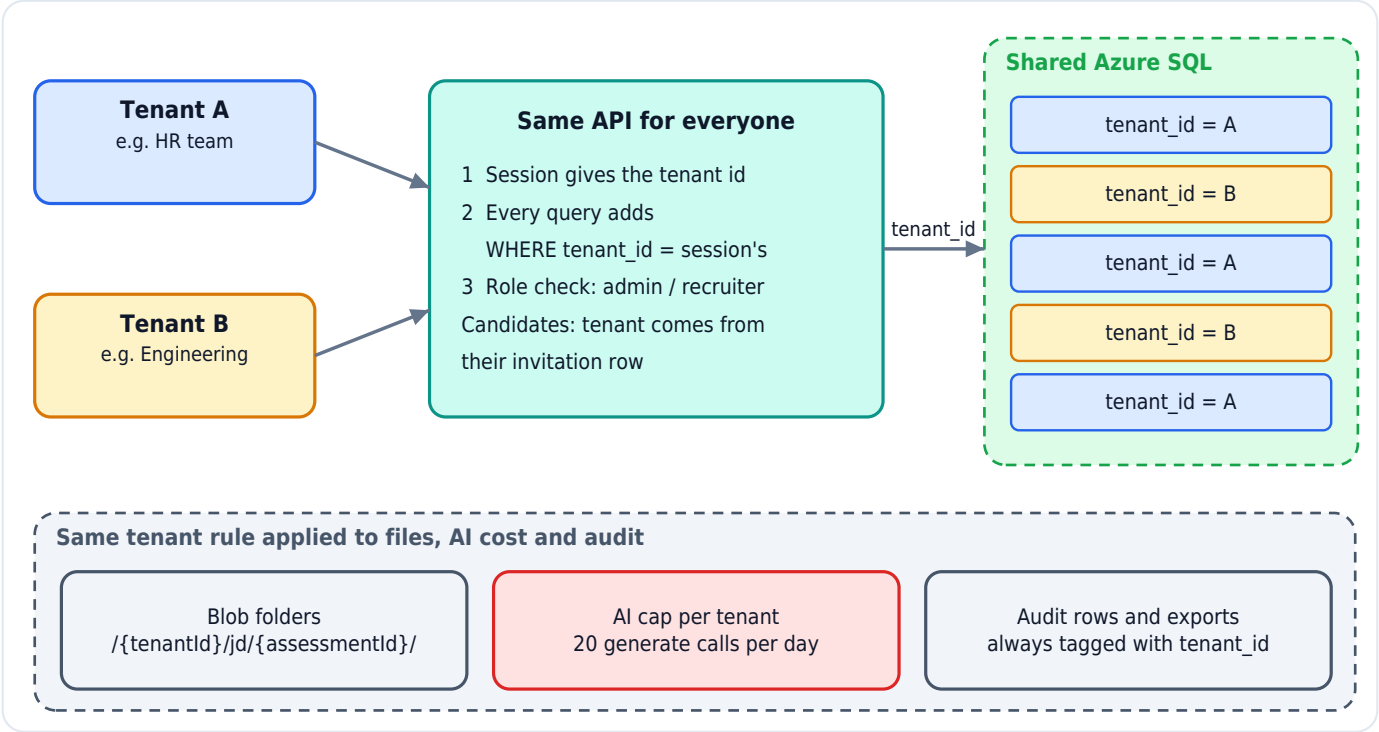


Figure 3. Tenant is read from the session, never from the request, and added to every query.

- Every table except `tenants` has `tenant_id`. All reads and writes go through repository functions that require a tenant id argument.
- Candidate calls use the invitation row to find the tenant, so a candidate can only reach their own attempt.
- New tenants are created by a protected call (`POST /platform/tenants` with a secret key) or a seed script. Each tenant gets a first Admin.
- Add one automated test: log in as Tenant A, request a Tenant B id, expect 404. Run it in GitHub Actions.

5. How frontend and backend talk to each other

Every screen calls one API group. The browser only talks to the web container; Next.js forwards `/api/*` to the api container.

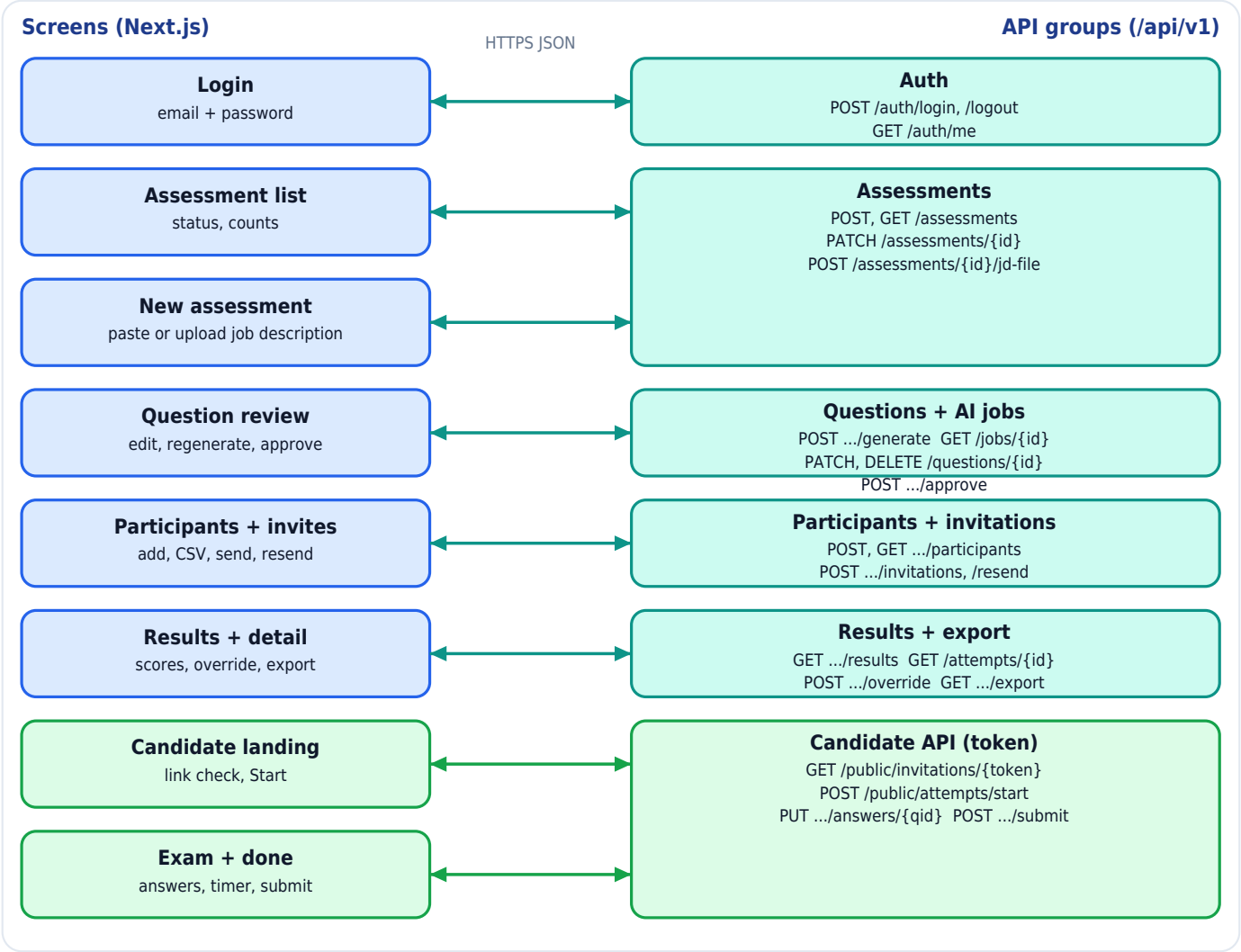
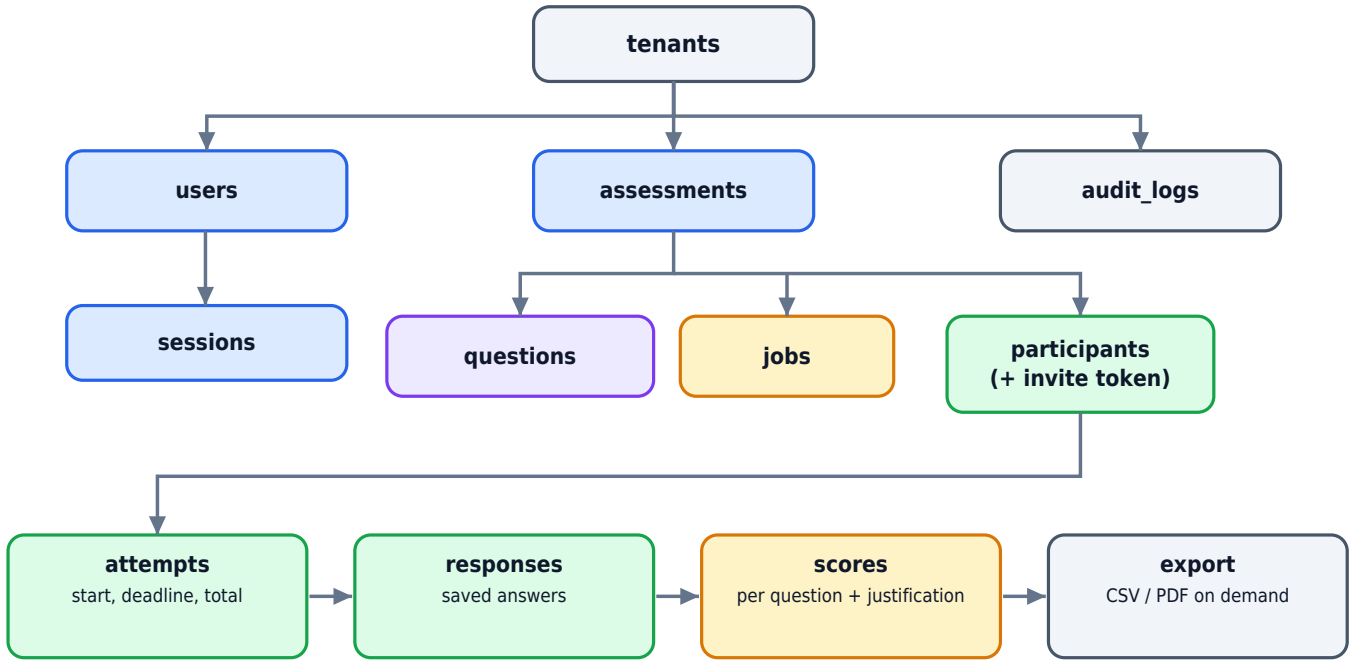


Figure 4. Each screen and the API group it uses. Candidate screens use a separate token-based API.

Rule	Detail
Transport	HTTPS, JSON, `/api/v1`, cookies sent automatically (same origin).
Login state	Cookie `sid` (HttpOnly, Secure, SameSite=Lax). On 401 the UI goes to /login.
Errors	`{"error":{"code":"JD_TOO_SHORT","message":"..."}}` with the right HTTP status. UI shows message.
Slow work	202 with `{jobId}`, then poll `GET /jobs/{id}` every 2 s until done or failed.
Candidate timer	Server sends `deadlineAt` and `serverTime`. Browser shows deadline minus server time. On 409 ATTEMPT_CLOSED the portal shows the submitted page.
Autosave	Every 30 s and on leaving a question. If a save fails, keep it in memory and retry every 10 s. Never discard answers.
Upload and download	JD file: multipart POST. Export: browser opens the export URL; the api returns the file.
Dates and ids	ISO 8601 UTC dates, UUID ids.

6. Data layer



Every table has tenant_id. Answer keys and rubrics stay in questions and never go to the candidate portal.

Figure 5. Data model. Everything hangs off the tenant; the bottom row is one candidate's path to a result.

Table	Main columns	Notes
tenants	id, name, status	Root
users	id, tenant_id, name, email, password_hash, role (admin/recruiter), status	Unique (tenant_id, email); bcrypt or Argon2
sessions	id_hash, user_id, tenant_id, role, expires_at	Cookie holds the random id; table holds its hash
assessments	id, tenant_id, title, job_description, jd_blob_path, duration_min, passing_score, sections_json, status, created_by	Status: draft, approved, open, closed
questions	id, tenant_id, assessment_id, type, body, options_json, correct, rubric, max_score, source, position, status	Answer key and rubric never sent to candidates
jobs	id, tenant_id, type (generate/mark/email), ref_id, status, attempts, error, tokens_used, created_at	The queue plus the daily AI count
participants	id, tenant_id, assessment_id, name, email, token_hash, token_expires_at, invite_status, send_attempts	Unique (assessment_id, email); token stored as hash
attempts	id, tenant_id, participant_id, started_at, deadline_at, submitted_at, status, total_score, max_score, percent	Status: in_progress, submitted, scored, needs_review
responses	attempt_id, question_id, value, saved_at	Unique (attempt_id, question_id), upsert
scores	attempt_id, question_id, score, max_score, source (auto/ai/manual), justification, status	Unique (attempt_id, question_id)
audit_logs	id, tenant_id, user_id, event, entity, entity_id, at	Create, approve, invite, submit, override, export

Blob Storage layout

```
container: jd-files   (private)
path:      {tenantId}/{assessmentId}/{originalFileName}
access:    only the api reads or writes; no public links
```

Data layer contract (functions the API code calls)

```
// every function takes tenantId first; it is added to the WHERE clause or INSERT
assessments.create(tenantId, userId, input)         -> assessment
assessments.list(tenantId, {status})                -> assessment[]
questions.listByAssessment(tenantId, assessmentId)  -> question[]
participants.addMany(tenantId, assessmentId, rows)  -> {added, duplicates, invalid}
attempts.start(tenantId, participantId)             -> attempt   // sets deadline_at
responses.upsert(tenantId, attemptId, questionId, value)  // rejects after deadline + 15 s
scores.save(tenantId, attemptId, questionId, score, source, justification)
jobs.enqueue(tenantId, type, refId) / jobs.claimNext(type) / jobs.finish(id, result|error)
```

Important indexes: `(tenant_id, assessment_id)` on questions, participants and attempts; unique `participants(token_hash)`; `(attempt_id, question_id)` on responses and scores; `jobs(status, type, created_at)`.

7. AI layer (Azure AI Foundry, GPT-4o)

The AI layer is one small module with two functions. It is only called by the job loop, never directly from a web request.

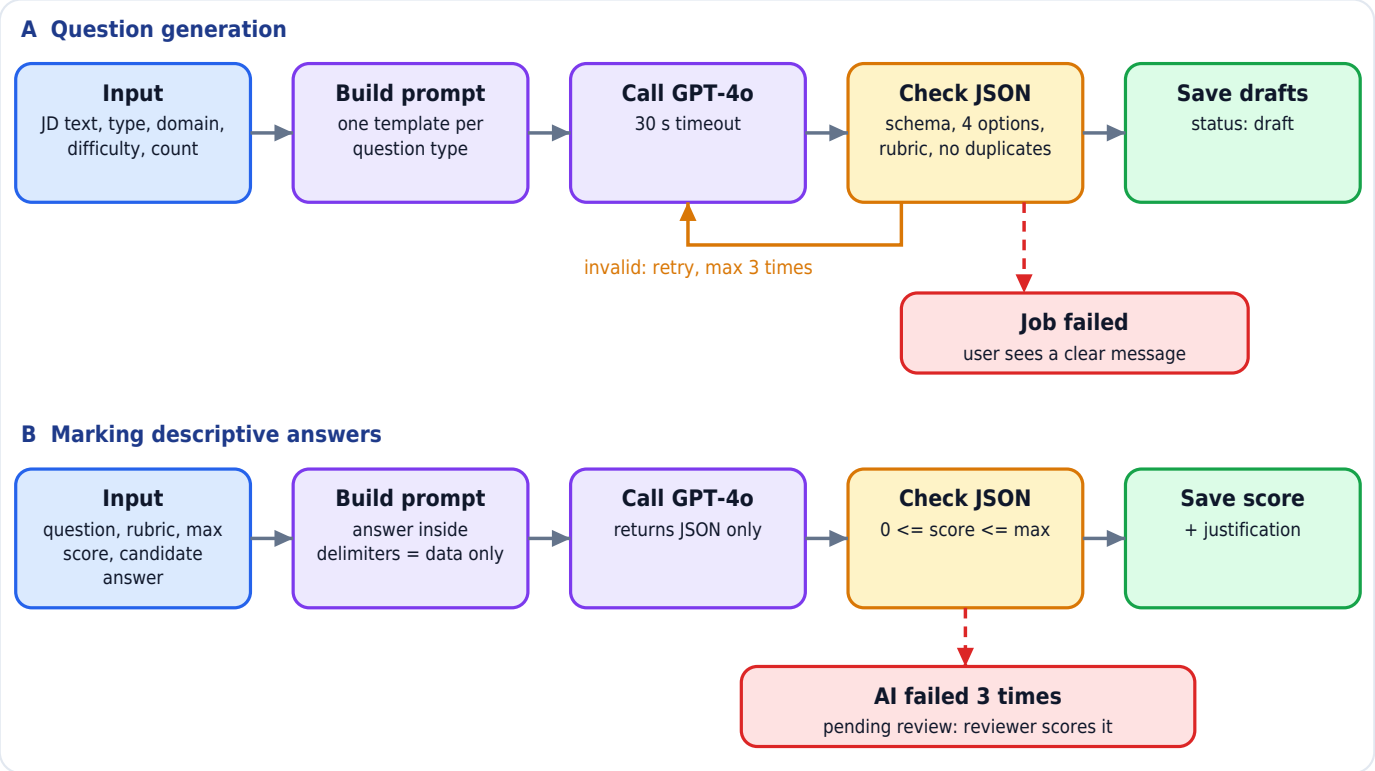


Figure 6. AI layer: generating questions (A) and marking descriptive answers (B).

Function contracts

```
generateQuestions({ jobDescription, type, domain, difficulty, count })
  -> Question[]            // throws AI_OUTPUT_INVALID after 3 tries

Question (mcq / cognitive):  { question, options: [A,B,C,D], correct: "A.."D", explanation }
Question (descriptive):      { question, rubric, maxScore, wordLimit }

markAnswer({ question, rubric, maxScore, answer })
  -> { score: number, justification: string }     // 0 <= score <= maxScore
```

Rule	Detail
Prompt	One template per type. It lists the JSON shape and says: return only JSON, no text around it.
Validation	Parse JSON, check required fields, MCQ has exactly 4 options and a valid correct letter, descriptive has a rubric, drop duplicates (over 80% word overlap).
Retry	Up to 3 attempts. On a retry the prompt adds: the previous answer was invalid, return valid JSON only. 429: wait 2 s then retry.
Timeout	30 s per call. On failure the job is marked failed with a user-readable message.
Marking safety	Candidate answer is placed between delimiters and the prompt says it is data, not instructions. Score is clamped to 0..max.

Rule	Detail
Privacy	Send only the answer text. Never send candidate name or email.
Cost control	Temperature 0.7 for generation, 0.2 for marking. Log tokens in jobs.tokens_used. Daily cap: 20 generate calls per tenant.
Human review	AI scores are advisory. Reviewers can override and the original AI score is kept.

8. Other layers: Blob, email, job loop

Layer	Contract (what the API code calls)	Behaviour
Blob Storage	files.saveJd(tenantId, assessmentId, file) returns {path}	Keeps the original. Text is extracted (pdf-parse or pypdf) and stored in assessments.job_description.
Email (ACS)	email.sendInvite({to, name, title, durationMinutes, link, expiresAt}) returns {messageId}	HTML template with a Start Assessment button. Failure sets invite_status = failed; the Resend button creates a new link.
Job loop	types: generate, mark, email	Runs every 2 s, claims up to 3 jobs, retries 3 times, re-queues stuck jobs, auto-submits late attempts every minute.
Audit	audit.log(tenantId, userId, event, entity, id)	One insert per important action. Used for support and disputes.

Candidate link: `https://{web}/a/{token}`. The token is a 256-bit random string; only its SHA-256 hash is stored. Opening the link only checks it; the exam begins when the candidate presses Start.

9. API contracts, one diagram per group

Base path `/api/v1`. Who: **any** = logged-in staff, **R+** = recruiter or admin, **admin** = admin only, **token** = candidate link, **attempt** = candidate attempt cookie. All staff calls need the session cookie, and the tenant always comes from it.

9.1 Auth

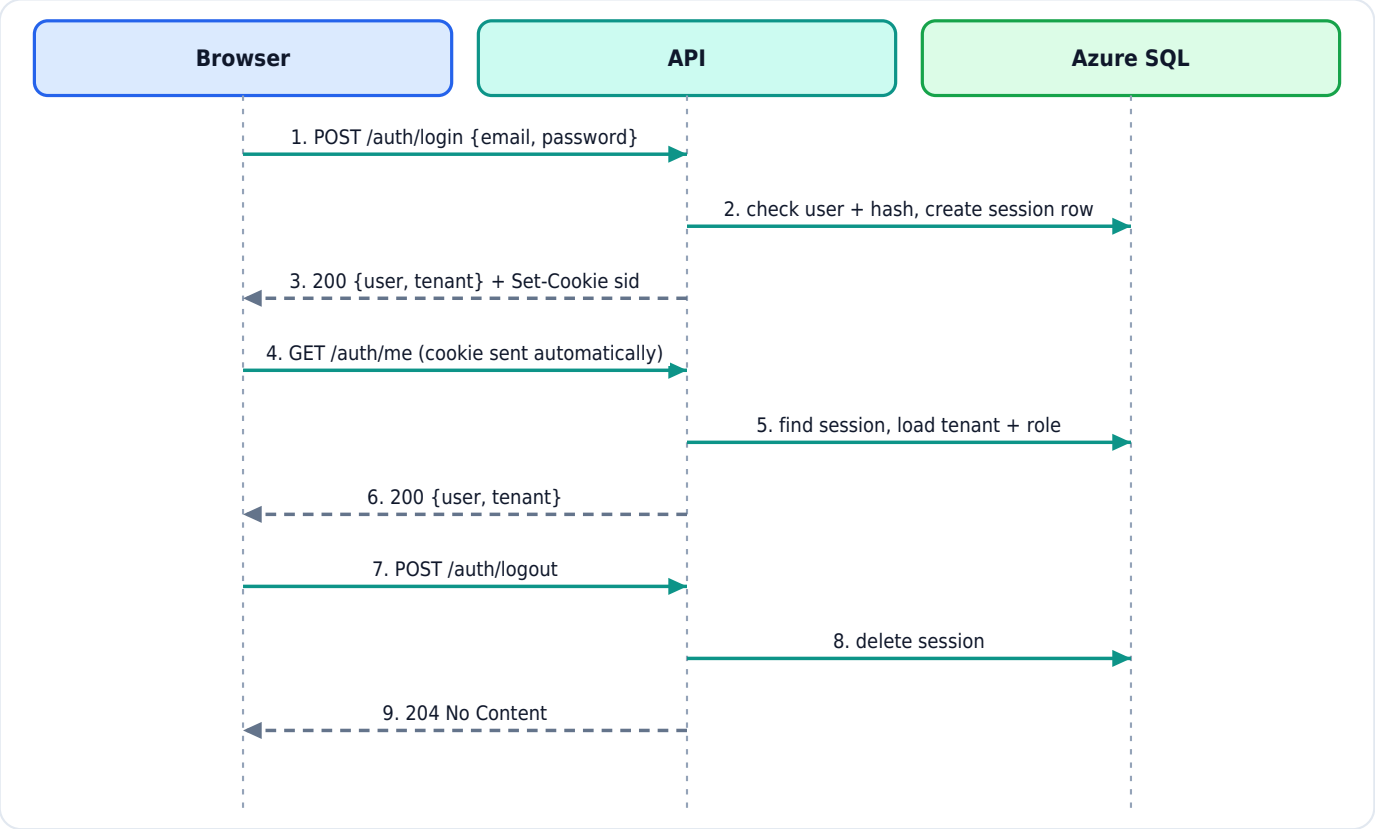


Figure 7. Auth contract: login creates a session, every call is checked against it.

Method	Path	Who	Request and response
POST	/auth/login	public	{email, password} returns 200 {user{id,name,role}, tenant{id,name}} + cookie. 401 INVALID_CREDENTIALS. 429 after 5 failures.
POST	/auth/logout	any	204. Session deleted.
GET	/auth/me	any	200 {user, tenant}. 401 if the session is missing or expired.

9.2 Tenants and users

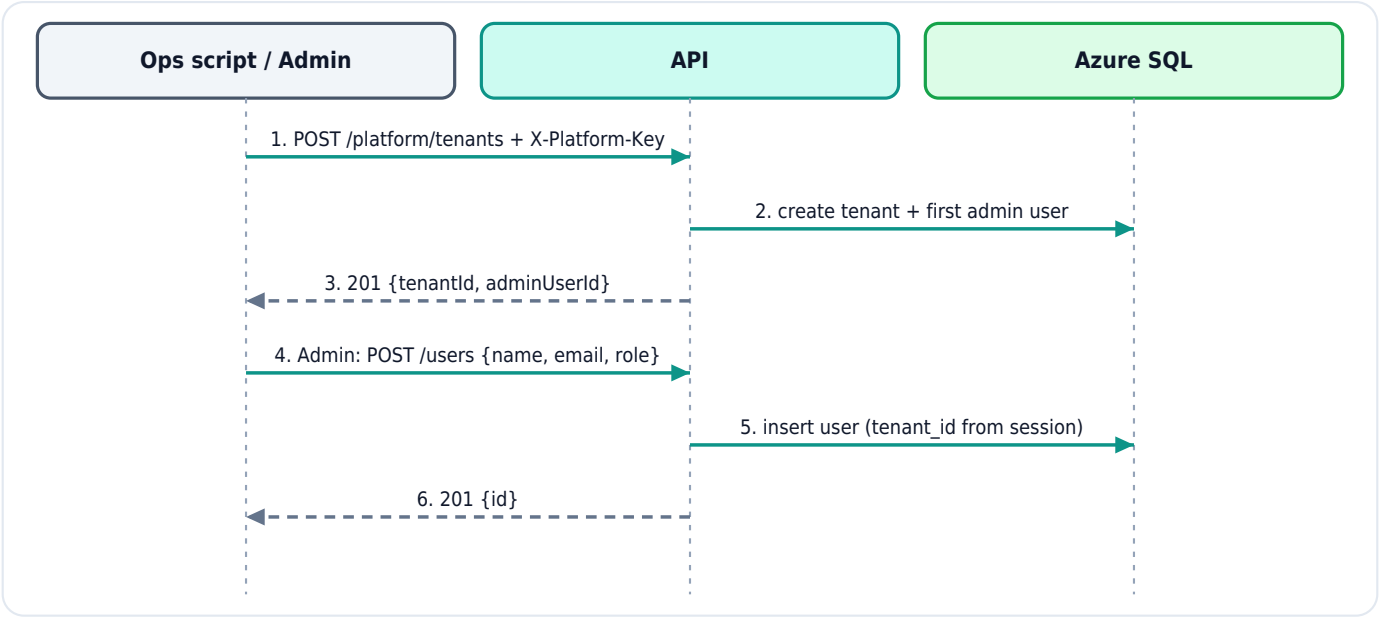


Figure 8. Tenant onboarding and user management.

Method	Path	Who	Request and response
POST	/platform/tenants	key	Header X-Platform-Key. {name, adminName, adminEmail, password} returns 201 {tenantId, adminUserId}.
GET	/users	admin	200 [{id, name, email, role, status}]
POST	/users	admin	{name, email, role, password} returns 201 {id}. 409 EMAIL_EXISTS.
PATCH	/users/{id}	admin	{role?, status?} returns 200. Disabling a user deletes their sessions.

9.3 Assessments and job description (paste or upload)

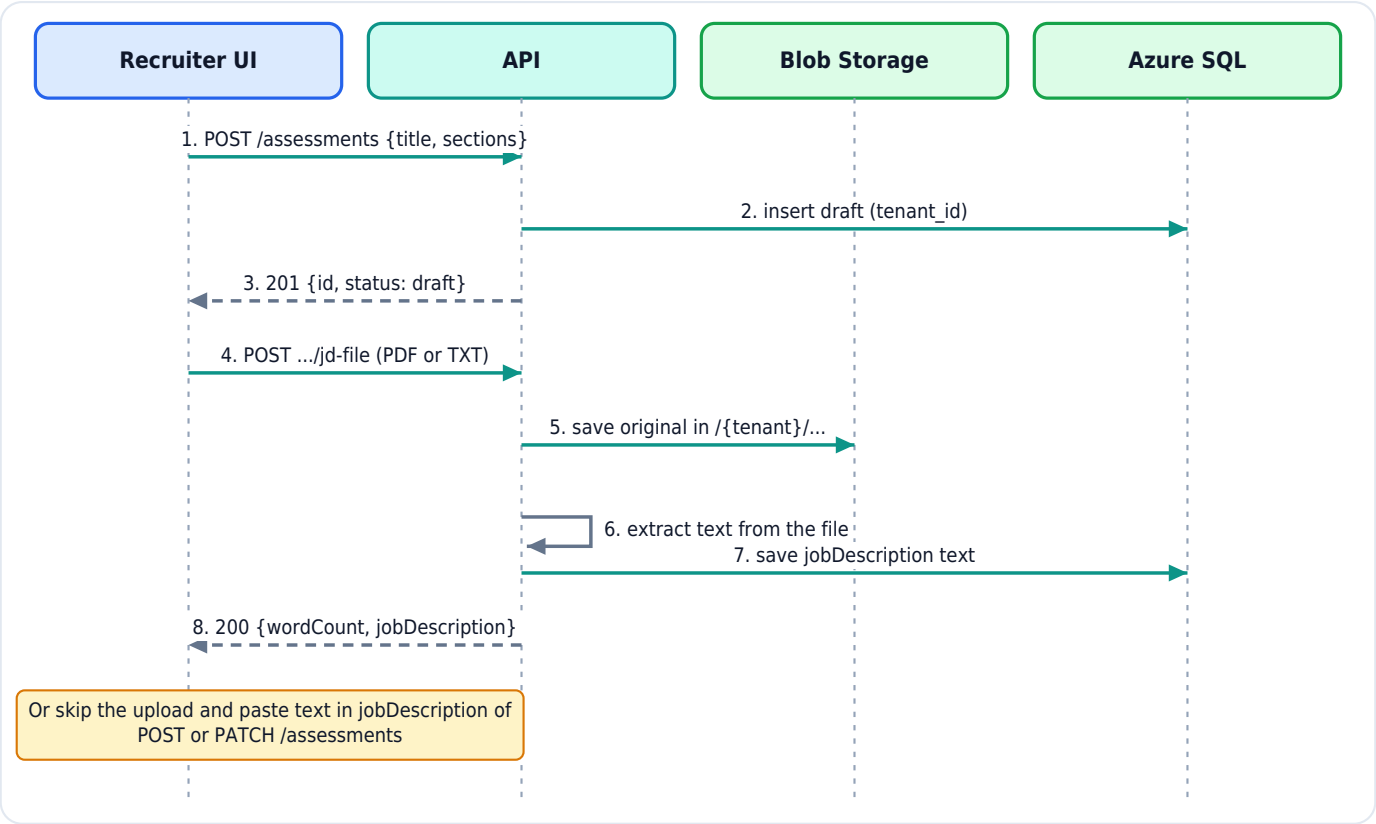


Figure 9. Assessment contract with both ways of giving the job description.

Method	Path	Who	Request and response
POST	/assessments	R+	{title, jobDescription?, durationMinutes, passingScore, sections[{type: mcq|descriptive|cognitive, domain, difficulty: easy|medium|hard, count}]} returns 201 {id, status:'draft'}.
GET	/assessments	R+	?status returns 200 [{id, title, status, participantCount, submittedCount, createdAt}].
GET	/assessments/{id}	R+	200 full assessment with sections, jobDescription, jdFileName.
PATCH	/assessments/{id}	R+	Edit title, duration, passingScore, jobDescription, sections while draft. 409 if approved.
POST	/assessments/{id}/jd-file	R+	multipart field file, .pdf or .txt up to 2 MB. 200 {fileName, wordCount, jobDescription}. 422 FILE_UNREADABLE.
POST	/assessments/{id}/close	R+	200. No new attempts; invitations expire.
DELETE	/assessments/{id}	R+	204. Draft only.

```
POST /api/v1/assessments
{ "title": "Backend Engineer", "jobDescription": "We are hiring ...", "durationMinutes": 60, "passingScore": 60,
  "sections": [ {"type":"mcq","domain":"Node.js","difficulty":"medium","count":10},
                {"type":"descriptive","domain":"System design","difficulty":"hard","count":3} ] }
-> 201 { "id": "5c1e...", "status": "draft" }
```

9.4 Generate questions (asynchronous job)

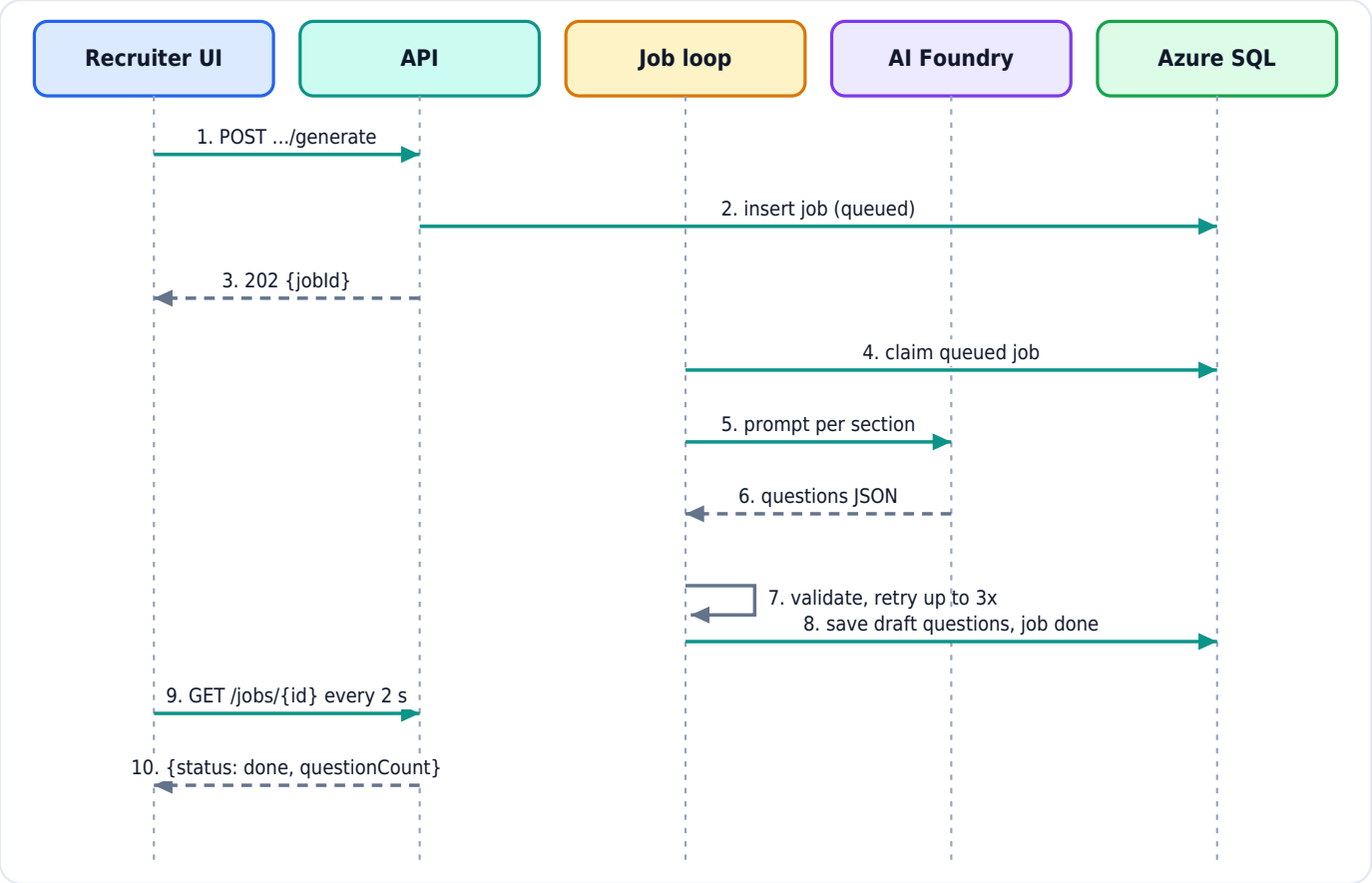


Figure 10. Generation contract: the API replies at once, the job loop does the AI work.

Method	Path	Who	Request and response
POST	/assessments/{id}/generate	R+	{sectionIndexes?: [0,1]} returns 202 {jobId}. 422 JD_TOO_SHORT (under 100 words). 429 DAILY_AI_LIMIT. 409 if approved.
GET	/jobs/{id}	R+	200 {status: queued|running|done|failed, error?, questionCount?}

9.5 Review and approve questions

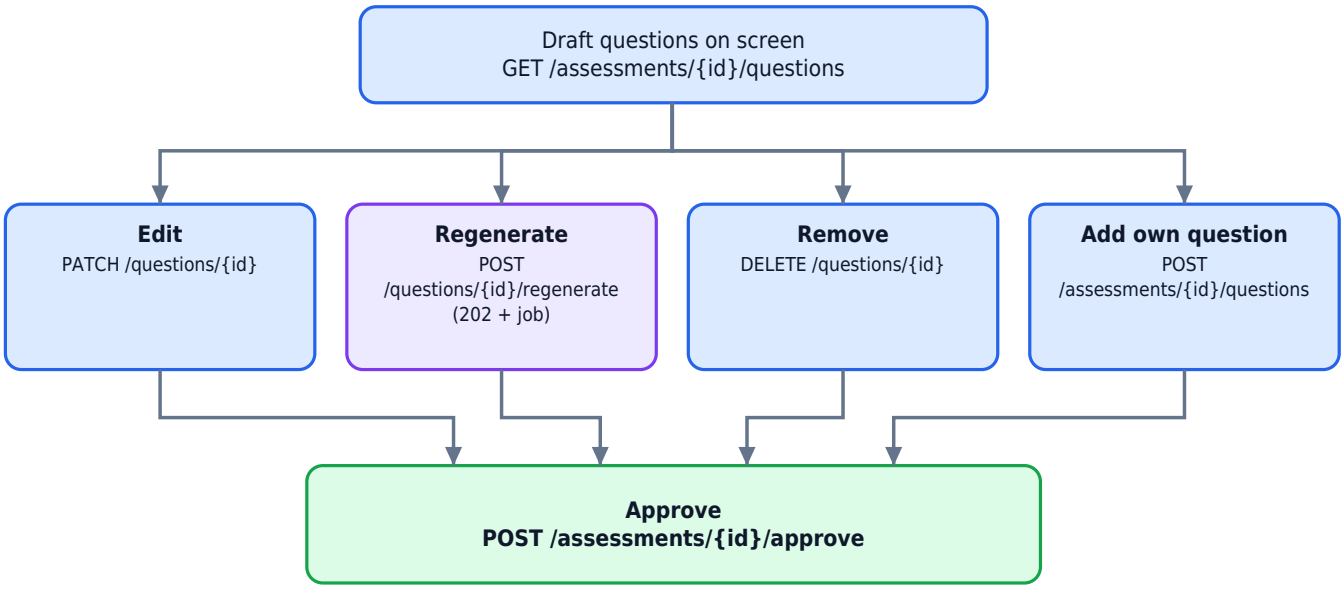


Figure 11. Review contract: four actions on draft questions, then one approval.

Method	Path	Who	Request and response
GET	/assessments/{id}/questions	R+	200 [{id, type, body, options?, correct?, rubric?, maxScore, source: ai|custom, position}]
POST	/assessments/{id}/questions	R+	Custom question {type, body, options?, correct?, rubric?, maxScore} returns 201.
PATCH	/questions/{id}	R+	Edit fields. 409 after approval.
DELETE	/questions/{id}	R+	204.
POST	/questions/{id}/regenerate	R+	202 {jobId}. Replaces that one question.
POST	/assessments/{id}/approve	R+	200 {status:'approved'}. 409 NO_QUESTIONS or SECTION_INCOMPLETE.

```
GET /api/v1/jobs/9a7b...        -> 200 { "status": "done", "questionCount": 13 }
Question: { "id":"q1", "type":"mcq", "body":"Which HTTP status ...?", "options":["200","201","204","409"],
            "correct":"B", "maxScore":1, "source":"ai", "position":1 }
```

9.6 Participants and invitations

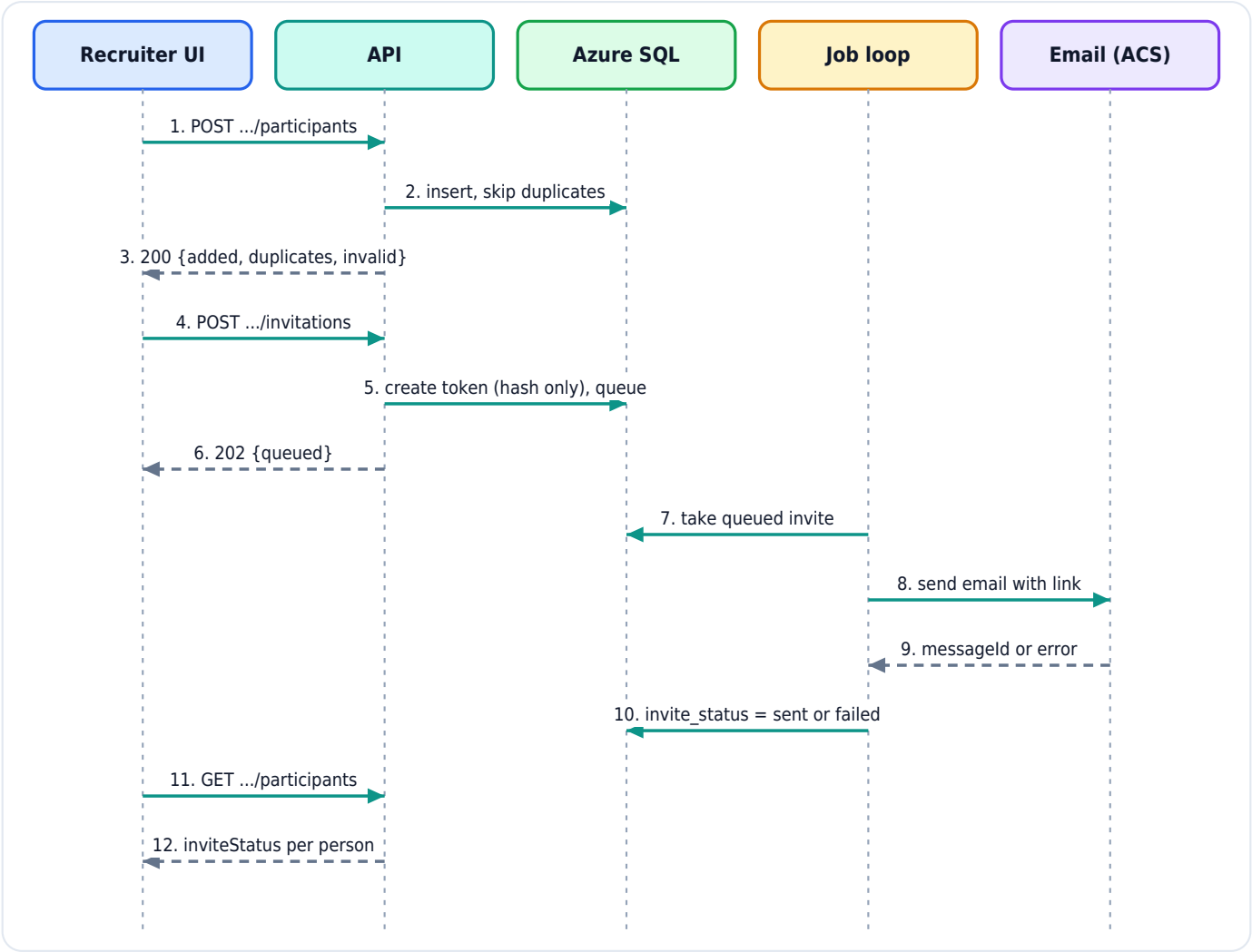


Figure 12. Participant contract: add people, queue invitations, the job loop sends the emails.

Method	Path	Who	Request and response
POST	/assessments/{id}/participants	R+	{participants:[{name, email}]} or {csv:'name,email\n...'} returns 200 {added, duplicates, invalid[]}. Max 200.
GET	/assessments/{id}/participants	R+	200 [{id, name, email, inviteStatus: none|queued|sent|failed, attemptStatus, percent?}]
DELETE	/participants/{id}	R+	204. Only before the attempt starts.
POST	/assessments/{id}/invitations	R+	{participantIds?} (default: all not yet invited) returns 202 {queued}. 409 NOT_APPROVED.
POST	/participants/{id}/resend	R+	202. New link, the old link stops working.

9.7 Candidate exam (token based)

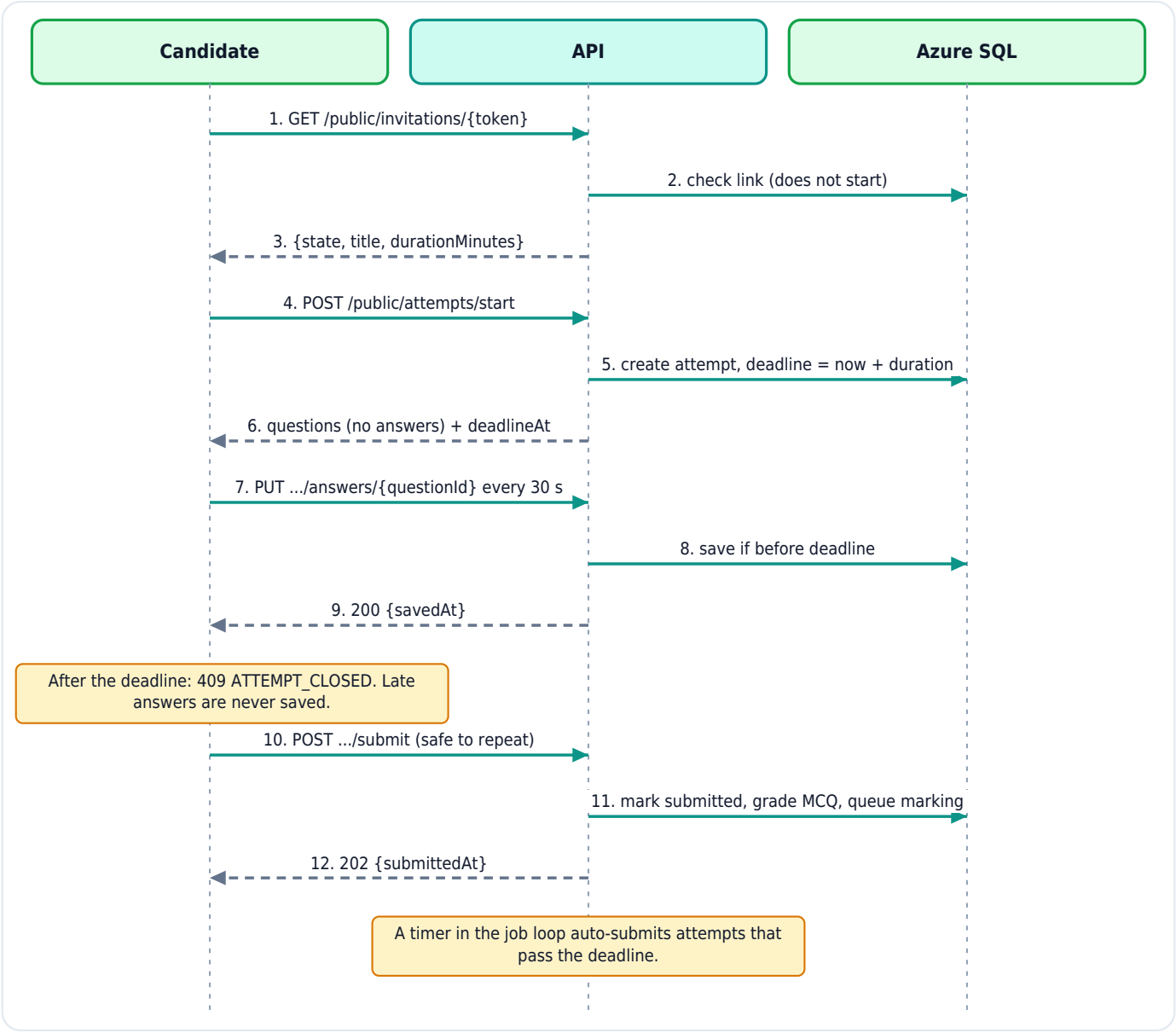


Figure 13. Candidate contract: link check, Start, autosave, submit.

Method	Path	Who	Request and response
GET	/public/invitations/{token}	token	200 {state: valid|expired|used, title, durationMinutes, instructions, companyName}. Read-only.
POST	/public/attempts/start	token	{token} returns 200 {attemptId, deadlineAt, serverTime, questions[], answers[]} + attempt cookie. Calling again resumes the same attempt.
GET	/public/attempts/current	attempt	Same shape as start. Used after a page refresh.
PUT	/public/attempts/current/answers/{questionId}	attempt	{value} returns 200 {savedAt}. 409 ATTEMPT_CLOSED after deadline + 15 s.
POST	/public/attempts/current/submit	attempt	202 {submittedAt}. A second call returns 200.

```
POST /api/v1/public/attempts/start   { "token": "kP3x..." }
-> 200 { "attemptId":"0f6c...", "deadlineAt":"2026-11-05T11:30:00Z", "serverTime":"2026-11-05T10:30:02Z",
         "questions":[ {"id":"q1","type":"mcq","body":"...","options":["...","...","...","..."],"maxScore":1},
                       {"id":"q2","type":"descriptive","body":"...","wordLimit":250,"maxScore":10} ],
         "answers":[] }
PUT  /api/v1/public/attempts/current/answers/q2   { "value": "I would start by ..." }   -> 200 { "savedAt": "..." }
```

Candidates never receive the fields `correct`, `rubric` or `explanation`.

9.8 Scoring (internal, no HTTP call)

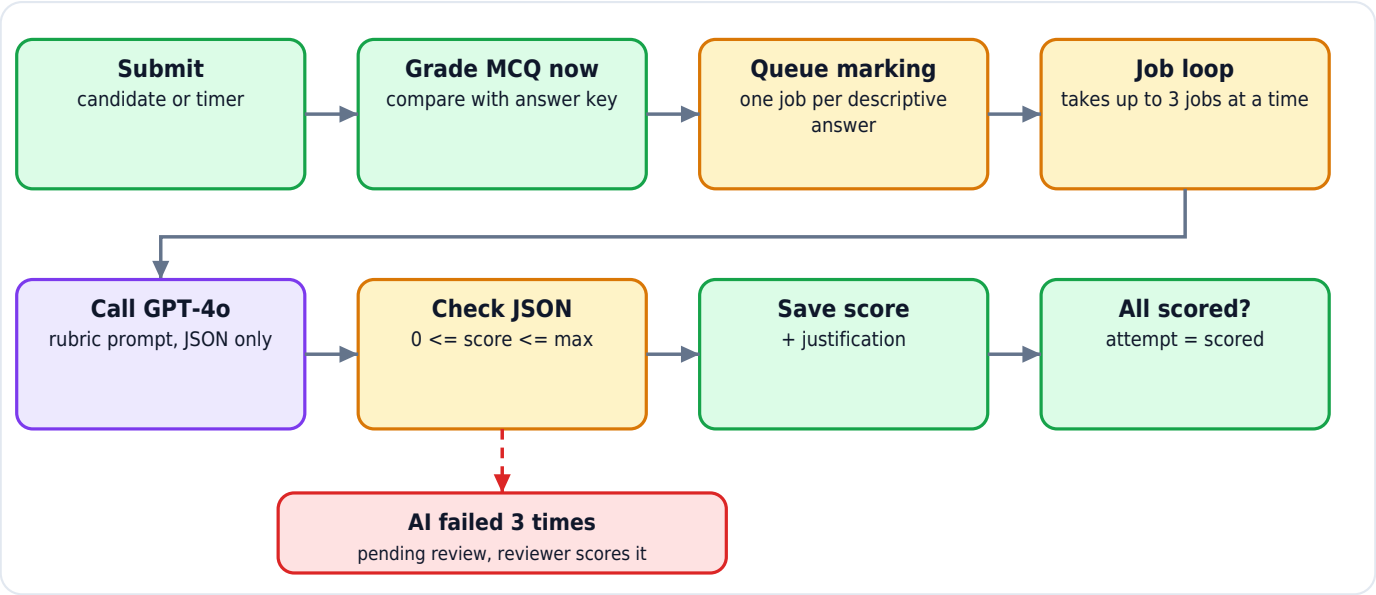


Figure 14. Scoring contract: MCQ is graded at submit, descriptive answers go through the job loop.

Scoring is triggered by submit and has no endpoint of its own. The attempt moves from `submitted` to `scored` when every question has a final score, or to `needs_review` if any AI marking failed.

9.9 Results, override and export

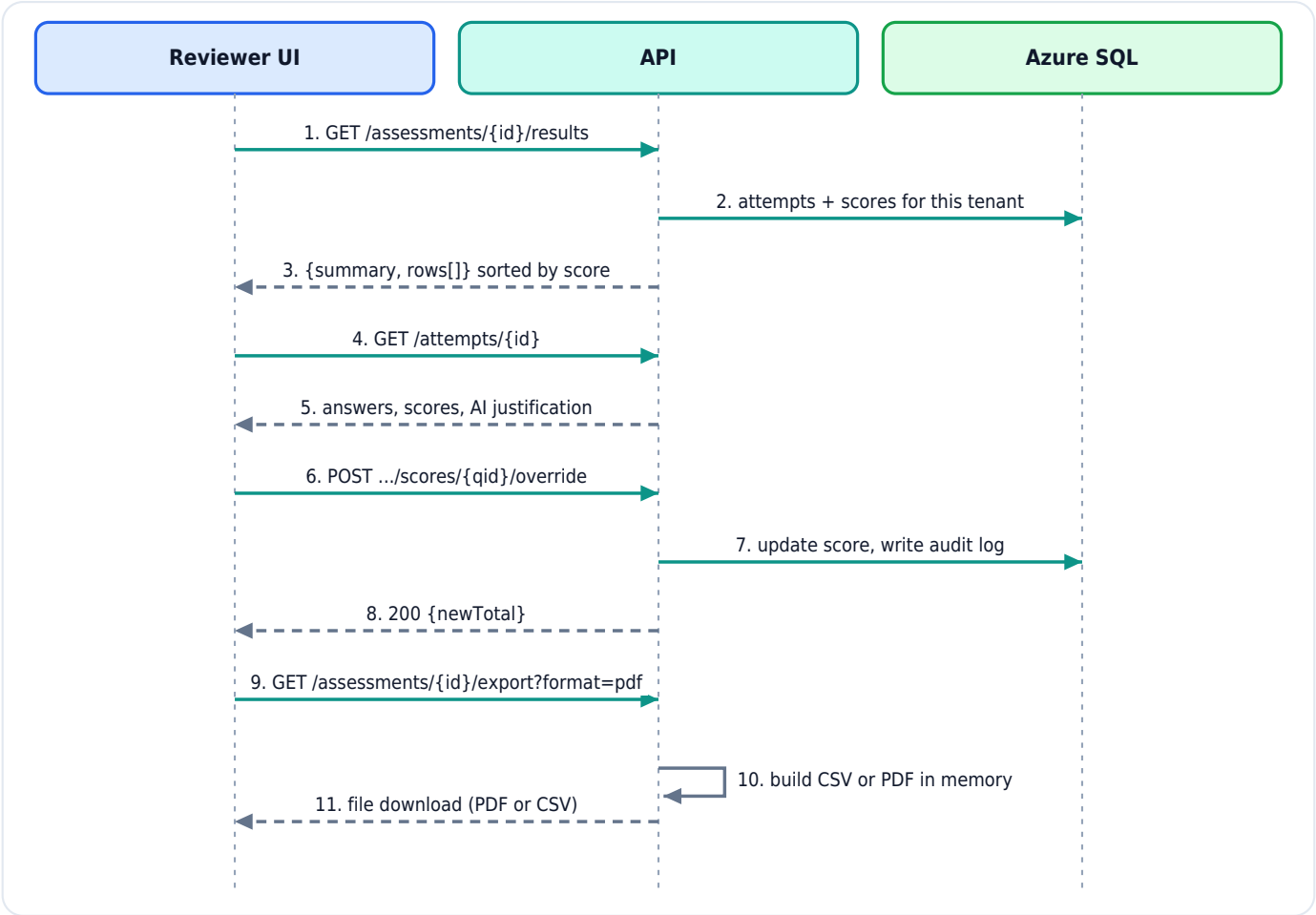


Figure 15. Results contract: list, detail, manual override, export.

Method	Path	Who	Request and response
GET	/assessments/{id}/results	R+	?sort=score|name returns 200 {summary{count, avg, passRate}, rows[{attemptId, name, email, score, maxScore, percent, passed, timeTakenSec, status}]}.
GET	/attempts/{id}	R+	200 {participant, answers[{questionId, type, body, answer, correct?, score, maxScore, source: auto|ai|manual, justification}]}
POST	/attempts/{id}/scores/{questionId}/override	R+	{score, comment} returns 200 {newTotal}. Original AI score is kept in the audit log.
GET	/assessments/{id}/export	R+	?format=csv|pdf returns the file with Content-Disposition. Columns: name, email, each section score, total, percent, time taken, status.
GET	/health	public	200 {status:'ok'}. Used by Container Apps and the smoke test.

Error codes

400 VALIDATION_ERROR, 401 UNAUTHENTICATED, 403 FORBIDDEN, 404 NOT_FOUND (also for another tenant's id), 409 CONFLICT, ATTEMPT_CLOSED or NOT_APPROVED, 410 LINK_EXPIRED, 422 JD_TOO_SHORT or AI_OUTPUT_INVALID, 429 RATE_LIMITED or DAILY_AI_LIMIT, 500 INTERNAL, 503 AI_UNAVAILABLE. Body: `{"error":{"code":"...","message":"...","details":[...]}}`.

10. Final API contract map

One page with every endpoint, who calls it, and which Azure service it touches.

Recruiter / admin screens (session cookie)

Auth — SQL

POST /auth/login POST /auth/logout
GET /auth/me

Platform + users — SQL

POST /platform/tenants (X-Platform-Key)
GET, POST /users PATCH /users/{id}

Assessments + job description — SQL, Blob

POST, GET /assessments
GET, PATCH, DELETE /assessments/{id}
POST /assessments/{id}/jd-file POST /assessments/{id}/close

Questions + AI jobs — SQL, AI

POST /assessments/{id}/generate GET /jobs/{id}
GET, POST /assessments/{id}/questions
PATCH, DELETE /questions/{id}
POST /questions/{id}/regenerate
POST /assessments/{id}/approve

Participants + invitations — SQL, Email

POST, GET /assessments/{id}/participants
DELETE /participants/{id}
POST /assessments/{id}/invitations
POST /participants/{id}/resend

Results + export — SQL

GET /assessments/{id}/results GET /attempts/{id}
POST /attempts/{id}/scores/{qid}/override
GET /assessments/{id}/export?format=csv|pdf

Health

GET /health

Candidate screens (link token)

Candidate API — SQL

GET /public/invitations/{token}
POST /public/attempts/start
GET /public/attempts/current
PUT /public/attempts/current/answers/{qid}
POST /public/attempts/current/submit

Rules on every call

Staff: cookie sid, tenant from the session
Candidate: link token, then attempt cookie
Error: {error:{code,message}}
AI / email work: 202 + poll GET /jobs/{id}

Chips mean the call touches

- SQL — Azure SQL
- Blob — Blob Storage
- AI — AI Foundry (via job loop)
- Email — Communication Svcs

Figure 16. All 37 endpoints grouped by caller. Chips show the Azure service behind each group.

11. CI/CD with GitHub Actions to Azure Container Apps

Pull requests run tests only. A merge to `main` builds both images, pushes them to Azure Container Registry, and updates the two container apps. GitHub signs in to Azure with OIDC, so no Azure password is stored in GitHub.

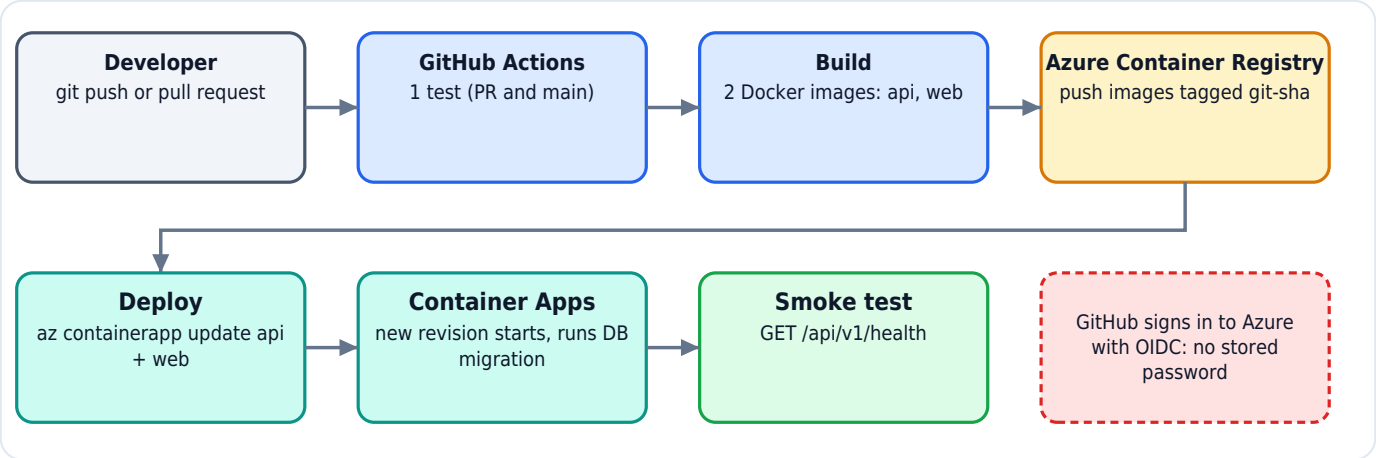


Figure 17. Deployment pipeline.

Starting workflow (`.github/workflows/deploy.yml`)

```
name: deploy
on:
  push:
    branches: [main]
  pull_request:
permissions:
  id-token: write
  contents: read
env:
  ACR: myassessacr          # registry name
  RG: rg-assessment-mvp     # resource group
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cd api && npm ci && npm test      # includes the cross-tenant test
      - run: cd web && npm ci && npm run build
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - run: az acr login --name $ACR
      - run: |
          docker build -t $ACR.azurecr.io/api:${{ github.sha }} ./api
          docker build -t $ACR.azurecr.io/web:${{ github.sha }} ./web
          docker push $ACR.azurecr.io/api:${{ github.sha }}
          docker push $ACR.azurecr.io/web:${{ github.sha }}
      - run: |
          az containerapp update -n assess-api -g $RG --image $ACR.azurecr.io/api:${{ github.sha }}
          az containerapp update -n assess-web -g $RG --image $ACR.azurecr.io/web:${{ github.sha }}
      - run: curl --fail https://${{ vars.WEB_URL }}/api/v1/health
```

One-time Azure setup for the pipeline

- Create the resource group, Container Registry, and a Container Apps environment with two apps: `assess-api` (internal ingress) and `assess-web` (external ingress).
- Give both container apps a managed identity with the **AcrPull** role on the registry.
- Create an app registration with a federated credential for your GitHub repo and branch main; give it **Contributor** on the resource group. Save its three ids as GitHub secrets.

- Store app settings as **Container Apps secrets** (table in section 12). Never commit them. Allow Azure services in the Azure SQL firewall.
- The api runs idempotent SQL migrations when it starts, so a deploy also updates the schema.

12. 4-day build plan, setup and risks

Day	Backend + AI	Frontend	Azure + DevOps
1	Schema with tenant_id, login + sessions, tenant seed, repository layer, job loop skeleton	Next.js shells: login, assessment list, layout; /api proxy	Create all Azure resources; repo; deploy a hello-world through GitHub Actions end to end
2	Assessments + JD upload (Blob + text extraction), generate job with Foundry, validator + retry, question endpoints	New assessment form (paste or upload), question review, approve	Foundry deployment, Blob container, secrets in Container Apps
3	Participants + CSV, invitations (ACS), attempt endpoints, deadline, autosave, submit, MCQ grading, AI marking jobs	Participants screen, candidate landing, exam page with timer and autosave, done page	ACS domain + sender, test real email, cross-tenant test in CI
4	Results, override, CSV + PDF export, audit log, limits, bug fixes	Results + detail screens, export buttons, loading and error states	App Insights, smoke test, demo data, full dry run on the deployed site

Environment variables (stored as Container Apps secrets)

Name	Used for
`DATABASE_URL`	Azure SQL connection string
`FOUNDRY_ENDPOINT, FOUNDRY_KEY, FOUNDRY_DEPLOYMENT`	GPT-4o calls
`ACS_CONNECTION_STRING, ACS_SENDER`	Invitation emails
`BLOB_CONNECTION_STRING, BLOB_CONTAINER`	Job description files
`SESSION_SECRET, PLATFORM_KEY`	Cookie signing; protects POST /platform/tenants
`APP_BASE_URL, API_INTERNAL_URL`	Links in emails; web to api proxy target

Main risks and the simple answer

Risk	Answer in the MVP
AI returns bad JSON	Validator + 3 retries; pre-generate demo questions before any live demo
AI marks unfairly	Advisory only; reviewer override; original score kept
Candidate loses answers	Autosave 30 s + browser retry queue + server deadline
One tenant sees another's data	Repository requires tenantId; automated cross-tenant test in CI
AI cost surprise	Daily generate cap per tenant, tokens logged in jobs
Single api replica restarts	Jobs live in SQL and are re-queued on start
Candidate data kept too long	Decide a retention period (for example 180 days) and delete after it

After the MVP: add candidate comparison, delivery webhooks, Redis cache, a staging environment, and Row-Level Security when more teams join.